

""

SMS Agent Starter App

Connects Twilio SMS to the Claude API so you can auto-reply to customer texts.

WHAT THIS DOES

1. A customer texts your Twilio number.
2. Twilio calls this app (a webhook) with the message.
3. This app sends the message to Claude, along with recent chat history for that phone number.
4. Claude's reply is sent back to the customer as a text.

SETUP STEPS

1. Install dependencies:

```
pip install flask anthropic twilio --break-system-packages
```
2. Get your keys:
 - Anthropic API key:

```
https://console.anthropic.com/  
  - Twilio Account SID + Auth Token:  
https://console.twilio.com/  
  - A Twilio phone number (buy one  
in the Twilio console)
```

3. Set environment variables (don't
hardcode keys in this file):

```
export ANTHROPIC_API_KEY="sk-  
ant-..."  
export  
TWILIO_AUTH_TOKEN="your_twilio_auth_t  
oken" (optional, for request  
validation)
```

4. Edit the SYSTEM_PROMPT below to
describe YOUR client's business
(services, hours, FAQs, tone).
This is what makes the bot useful
for a specific business.

5. Run locally to test:

```
python sms_agent_app.py
```

6. Deploy (so Twilio can reach it
over the internet). Easiest options:

- Render.com (free tier, connect
your GitHub repo)
- Railway.app (free/cheap tier)

7. In the Twilio console, set your phone number's "A MESSAGE COMES IN" webhook to:
 https://your-deployed-app-url.com/sms
 Method: HTTP POST

That's it – texts to that number will now get AI replies.

"""

```
import os
from collections import defaultdict

from flask import Flask, request
from anthropic import Anthropic
from twilio.twiml.messaging_response
import MessagingResponse

app = Flask(__name__)

# ---- CONFIG: customize this per
client/business ----
SYSTEM_PROMPT = """You are a helpful
SMS assistant for [BUSINESS NAME].
Business info:
- Hours: [e.g. Mon-Sat 9am-6pm]
- Services: [list services]
```

- Location: [address]
- Booking: [how customers book, e.g. "reply BOOK and a day/time"]

Keep replies short (this is SMS) – 1-3 sentences max.

Be warm and professional. If you don't know an answer, say a team member will follow up, and don't make up information.

"""

```
MODEL = "claude-sonnet-4-6"
```

```
MAX_HISTORY_MESSAGES = 10 # how many  
past messages to remember per phone  
number
```

```
anthropic_client = Anthropic() #  
reads ANTHROPIC_API_KEY from  
environment
```

```
# Simple in-memory conversation  
store: { phone_number: [ {role,  
content}, ... ] }
```

```
# NOTE: this resets if the app  
restarts. For production, swap this  
for a
```

```
# real database (e.g. SQLite,  
Postgres, or Redis) so history
```

```

persists.
conversations = defaultdict(list)


def get_claude_reply(phone_number:
str, incoming_text: str) -> str:
    """Send the message + recent
history to Claude and return its
reply."""
    history =
conversations[phone_number]
    history.append({"role": "user",
"content": incoming_text})

    # Keep only the most recent N
messages to control cost
    trimmed_history = history[-
MAX_HISTORY_MESSAGES:]

    response =
anthropic_client.messages.create(
        model=MODEL,
        max_tokens=300,
        system=SYSTEM_PROMPT,
        messages=trimmed_history,
    )

    reply_text = "".join(
        block.text for block in

```

```

response.content if block.type ==
"text"
    ).strip()

    history.append({"role":
"assistant", "content": reply_text})
    conversations[phone_number] =
history

    return reply_text or "Sorry, I
couldn't come up with a reply just
now."

@app.route("/sms", methods=["POST"])
def sms_webhook():
    """Twilio calls this URL every
time someone texts your number."""
    incoming_text =
request.form.get("Body", "").strip()
    from_number =
request.form.get("From", "unknown")

    if not incoming_text:
        reply = "Sorry, I didn't
catch a message. Could you try
again?"
    else:
        try:

```

```
        reply =
get_claude_reply(from_number,
incoming_text)
        except Exception as e:
            print(f"Error generating
reply: {e}")
            reply = "Sorry, something
went wrong on our end. Please try
again shortly."

        twiml = MessagingResponse()
        twiml.message(reply)
        return str(twiml), 200,
{"Content-Type": "text/xml"}

@app.route("/", methods=["GET"])
def health_check():
    return "SMS agent is running.",
200

if __name__ == "__main__":
    # For local testing only. Use a
production server (gunicorn) when
deployed.
    app.run(host="0.0.0.0",
port=5000, debug=True)
```

